Development of Finance Management Application with AI Assistant

By

D. Boranbay, D. Nygmetollauly, A. Galymzhan

Department of Computational and Data Science
Astana IT University

6B06101 — Computer Science
Supervisor: Anar Rakhimzhanova

June 2025
Astana

Dedication and Acknowledgements

This work is dedicated to **Anar Rakhimzhanova**, whose guidance, expertise, and unwavering support have been instrumental in shaping the direction and quality of this thesis. Her insightful feedback, encouragement, and patience throughout every stage of this project have been invaluable. We are deeply grateful for the opportunity to learn under her supervision.

Author's declaration

I declare that the work in this thesis was carried out in accordance with the requirements of the University's Regulations and that it has not been submitted for any other academic award. Except where indicated by specific references in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the thesis are those of the author.

SIGNED: ..... DATE: .....

SIGNED: ..... DATE: .....

SIGNED: ..... DATE: .....

List of Tables

Table **Page**

3.1 Core entities in the FiTrack data schema and their functional purpose. . . . 21
3.2 Key technologies and their roles in the project. 28

4.1 Overview of available API endpoints and their functionality. 41

List of Figures

Figure **Page**

1.1 Methods Used for Tracking Personal Expenses 5
1.2 Aspects of Personal Finances Currently Tracked by Users 5
1.3 Most Desired Automated Features in Financial Apps 6
1.4 User Attitudes Toward Automatic Receipt Scanning 6
1.5 Preference for Combined Expense and Net Worth Tracking 7
1.6 Main Frustrations with Current Tracking Methods 8
1.7 Expense Categories Users Find Hardest to Track 8
1.8 Frequency of Receiving PDF Bank Statements 9
1.9 User Methods for Extracting Data from PDF Bank Statements 9
1.10 Types of Financial Advice Users Would Trust from AI Systems 10

3.1 Overview of the Development Process 17
3.2 Service-Based MVP Architecture with Frontend, Backend, and Bank Statement Parsing Service 20
3.3 Entity-Relationship Diagram (ERD) for Core Financial Models 22

4.1 Dashboard on web (left) and mobile (right). 33
4.2 Asset and Liability Management page on the web version. 34
4.3 Mobile view: Asset table (left) and Liability table (right). 34
4.4 Chatbot interface on web (left) and mobile (right). 35
4.5 Period selection feature in the chatbot interface, allowing users to analyze transactions over the last 7, 30, or 90 days. 36
4.6 User interface for uploading a PDF bank statement in the FiTrack application. 37
4.7 Web version: Login page (left) and Sign Up page (right). 37
4.8 Mobile version: Sign Up page. 38
4.9 Web version: Profile management page with options to edit user data and change password. 39

4.10 Full view of the Swagger API documentation. 40

Table of Contents

Dedication and Acknowledgements **i**

Author's declaration **ii**

List of Tables **iii**

List of Figures **iv**

1 Introduction **1**

1.1 Literature Review 2

1.2 Research Context 3

1.3 Problem Definition 4

1.4 Relevance of the Topic 10

1.4.1 Technological Relevance 10

1.4.2 Societal Relevance 11

1.4.3 Practical Relevance 11

1.5 Summary 12

2 FiTrack App **13**

2.1 Aims of the Work 13

2.2 Novelty of the Project 13

2.3 Relevance of the Project 14

2.4 Objectives 15

3 Methods **17**

3.1 Data Collection and Market Analysis 18

3.1.1 Objectives of the Survey 18

3.1.2 Survey Format and Method 18

3.1.3 Participant Profile 19
3.1.4 Justification of Approach 19
3.2 MVP Design 19
3.2.1 MVP Architecture 19
3.2.2 Database Design 21
3.3 Agile Implementation 22
3.3.1 Backend 22
3.3.2 Frontend 26
3.3.3 Bank Statement Parsing Service 28
3.4 Quality Assurance 29
3.5 Deployment 30
3.6 Summary 31

4 Results **32**
4.1 Dashboard 32
4.2 Asset and Liability Management 33
4.3 Chatbot Assistant 35
4.4 Bank Statement Parsing 36
4.5 User Authentication: Login and Sign Up 37
4.6 Profile Management 38
4.7 API Documentation Overview 39
4.8 Summary 41

5 Discussion **42**
5.1 Limitations 42
5.2 Future Directions 43

6 Conclusion **45**
6.1 Key Achievements 45
6.2 Practical Implications 46
6.3 Final Remarks 46

A User Survey: Financial Management and Expense Tracking **47**

Bibliography **53**

Chapter 1

Introduction

In the context of today's digital economy, individuals encounter growing complexity in managing their personal finances. The widespread adoption of electronic payments, the rise of recurring subscription services, and the fragmentation of financial data across various platforms make it increasingly difficult to obtain a comprehensive view of one's financial status. Traditional methods—such as spreadsheets or basic budgeting apps—remain common, but they require significant manual effort and offer minimal personalization. As a result, users often experience inefficiencies, reduced financial awareness, and suboptimal decision-making. Existing personal finance applications aim to address these challenges but typically do so only partially. Many focus either on expense tracking or on net worth monitoring, seldom providing both in an integrated way. Automation capabilities are often limited to rule-based logic or semi-manual input, leaving users to transcribe transactions from bank statements themselves. Additionally, the insights these applications offer are usually generic and fail to reflect the user's unique financial behaviors or goals. Recent advances in artificial intelligence (AI) and web technologies offer an opportunity to rethink personal finance management. Tools powered by natural language processing (NLP) and machine learning can enable more personalized, dynamic, and user-friendly experiences. However, most consumer finance applications still underutilize these technologies, and genuinely comprehensive, AI-enabled platforms remain rare. This thesis introduces FiTrack, an intelligent personal finance platform developed to address these gaps. FiTrack combines three core features in a single system: automated extraction of transactions from bank statements and PDF files, a conversational AI assistant capable of delivering personalized financial insights, and a unified interface for tracking both expenses and net worth. In addition to presenting the technical architecture and implementation of

the platform, this work evaluates its usability and practical value through real user feedback. The remainder of this chapter provides an overview of existing personal finance tools, reviews relevant literature and industry developments, identifies existing gaps, and outlines how FiTrack contributes to the field.

1.1 Literature Review

Personal financial management has long been a critical aspect of individual well-being, influencing quality of life, decision-making, and long-term planning. Historically, people relied on handwritten ledgers or offline spreadsheets to track expenses and manage budgets. Although such methods provided a certain degree of control, they were labor-intensive and prone to human error. With the rise of smartphones and internet connectivity, mobile applications emerged as popular tools for budgeting and expense tracking. While these tools introduced greater convenience, they often required manual input and lacked the intelligence to adapt to user behavior or financial contexts. Even today, many widely used apps depend on static rules or user-defined categories, which can limit their effectiveness over time. In contrast, artificial intelligence offers the potential to revolutionize how financial data is processed, interpreted, and presented. Recent developments in natural language processing (NLP), machine learning, and cloud computing enable systems to analyze large volumes of unstructured data, identify spending patterns, and provide actionable insights with minimal user effort. AI assistants can support users by answering questions, issuing reminders, or helping them make sense of their financial habits. According to a 2023 report by Medius, AI integration in financial operations can reduce manual effort by up to 30%, improve accuracy in transaction categorization, and enhance user engagement [10]. Moreover, the growing acceptance of conversational AI platforms such as ChatGPT, Gemini, and others signals a shift in user expectations - people are increasingly comfortable interacting with digital systems in a conversational manner [4, 17]. Despite these advancements, most popular finance applications remain limited in scope. They often lack true automation and typically do not support features like parsing PDF bank statements or delivering dynamic recommendations. Many tools focus solely on either short-term budgeting or long-term investment tracking, requiring users to alternate between platforms. A analysis of existing applications highlights several common limitations: Mint provided basic budgeting tools and automatic transaction categorization but lacked advanced forecasting and net worth tracking [7]. Its features were eventually absorbed into Intuit's Credit Karma [12].

You Need A Budget (YNAB) is effective for disciplined, zero-based budgeting but lacks automatic transaction imports in its basic version and has no AI-driven insights or integrated net worth management [14, 25].
Empower (formerly Personal Capital) excels in investment analytics but offers minimal support for budgeting or daily cash flow analysis, limiting its use for holistic management [2].
Spendee features a polished interface with multi-currency support but struggles with synchronization and lacks personalization in analytics [18].
This literature and market analysis underscores the limitations of existing financial applications. Despite incremental improvements in user interfaces and integration features, most tools fail to offer intelligent automation, contextual insights, or support for unstructured data sources like PDF bank statements. As user expectations shift toward more proactive and personalized financial guidance, the absence of AI-driven features and true end-to-end automation highlights a significant innovation gap in the current landscape. This creates a clear opportunity for developing next-generation solutions that combine convenience, intelligence, and adaptability for modern personal finance management.

1.2 Research Context

This research is situated at the intersection of personal finance, automation, AI, and user-centered software design. Although the fintech industry has made notable progress in digitizing financial services, most available tools solve only isolated aspects of the personal finance problem. For example, while some platforms offer solid expense categorization, they do not address the common user task of manually transcribing transactions from external statements—especially those in PDF format. Another major limitation lies in the fragmentation of tools. Users who wish to monitor their income, expenses, assets, liabilities, and net worth often must rely on multiple applications, each covering only part of their financial landscape. Furthermore, personalization—commonly found in sectors such as e-commerce and entertainment—remains underdeveloped in consumer finance. Most financial apps do not offer context-aware recommendations or insights tailored to individual behaviors. FiTrack was developed to address these shortcomings through three key innovations:

Seamless automation: Automatic extraction and categorization of transactions

from bank statements and PDFs, reducing manual effort.

Conversational AI: A GPT-4o mini-powered chatbot that delivers personalized financial insights, summaries, and recommendations.

Unified management interface: A single platform that combines expense tracking, income management, and net worth analysis.

By integrating these capabilities into one system, FiTrack aims to redefine user expectations of personal finance applications, providing a smarter, more efficient, and more user-centric experience.

1.3 Problem Definition

Despite the growing availability of personal finance applications, a significant proportion of users continue to rely on manual or semi-manual methods to track their financial activities. Data collected through user surveys for this project reveal a fragmented and inefficient landscape in personal finance management, characterized by limited automation, lack of integration between key financial dimensions, and poor user experience in existing tools.

The majority of respondents currently track their expenses using manual tools. Mobile applications were the most common method (265 responses), followed by spreadsheets (135 responses), while only 112 users utilize automated bank connections. A small but notable group (27 participants) reported not tracking finances at all, underscoring an accessibility gap in the market (Figure 1.1).

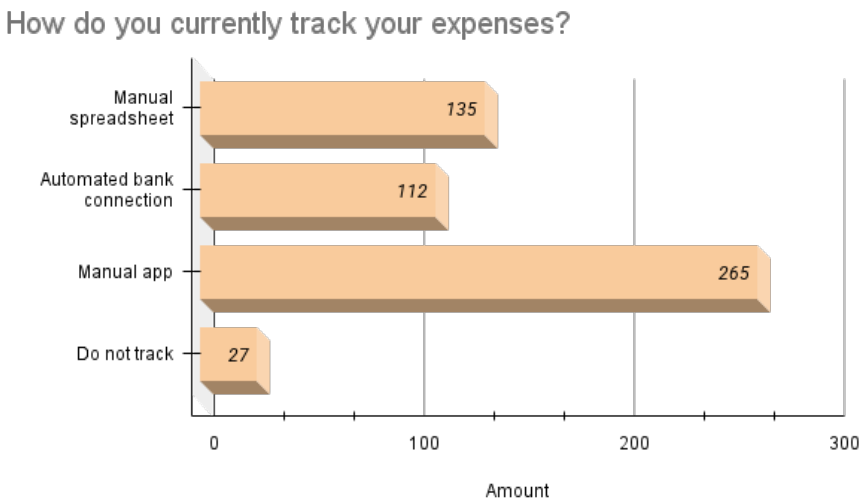

Figure 1.1: Methods Used for Tracking Personal Expenses

Furthermore, the scope of financial tracking remains narrow for most users. Although 25.2% track only expenses, far fewer users monitor income or net worth in conjunction. Only 20.6% track both expenses and net worth, and just 4.8% focus solely on net worth. This suggests that most tools encourage isolated tracking of financial metrics rather than providing a unified view (Figure 1.2).

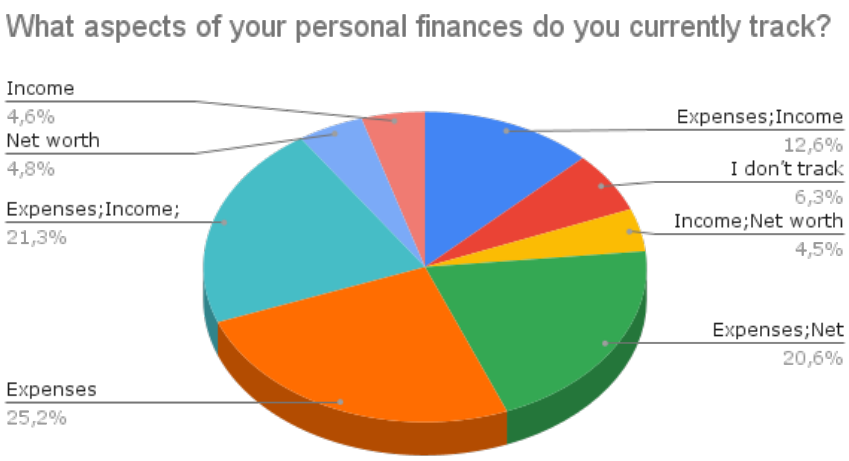

Figure 1.2: Aspects of Personal Finances Currently Tracked by Users

Automation appears to be a highly sought-after improvement. Among the tasks

users wish to automate, the most requested feature is bank statement parsing (30.2%), followed by automatic report generation (26.9%) and intelligent budget limits (22.3%) (Figure 1.3).

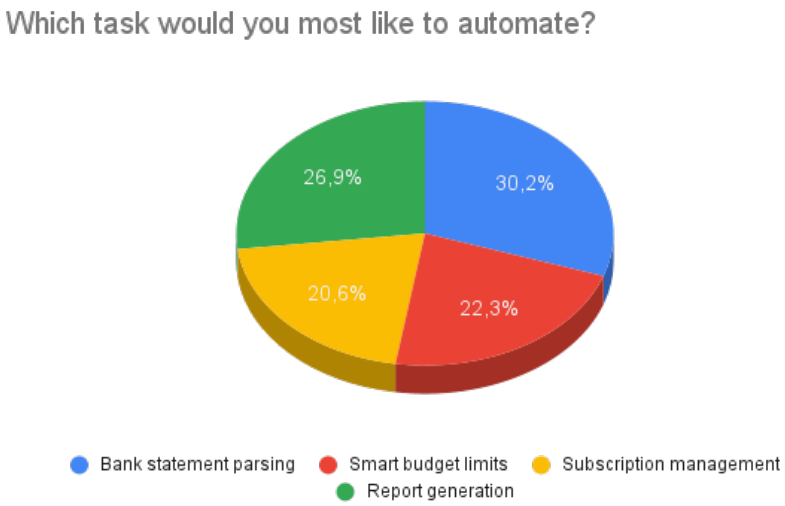

Figure 1.3: Most Desired Automated Features in Financial Apps

This interest in automation extends to receipt scanning. Over half of the users (50.3%) believe that automatic receipt scanning would save them time, while another 29.7% support it if accuracy can be guaranteed - demonstrating a clear expectation for intelligent, accurate input mechanisms (Figure 1.4).

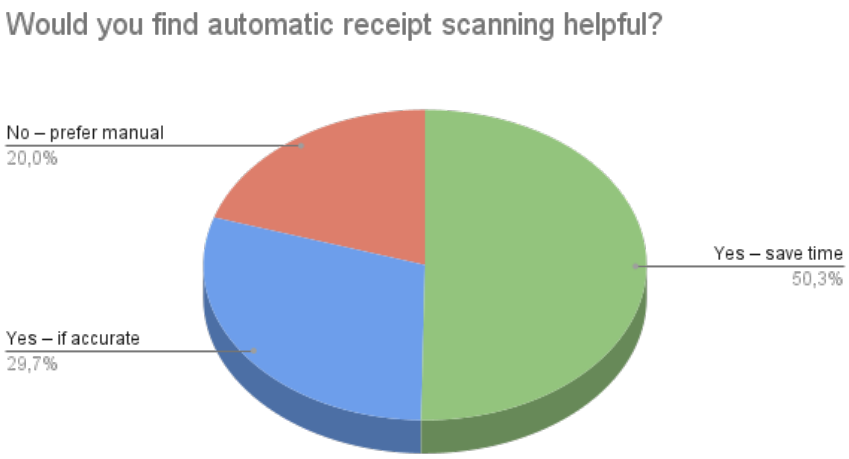

Figure 1.4: User Attitudes Toward Automatic Receipt Scanning

In addition, when asked whether they would prefer a unified application combining expense and net worth tracking, an overwhelming 82% of users answered affirmatively, reinforcing the necessity of integrating multiple dimensions of financial management in a single platform (Figure 1.5).

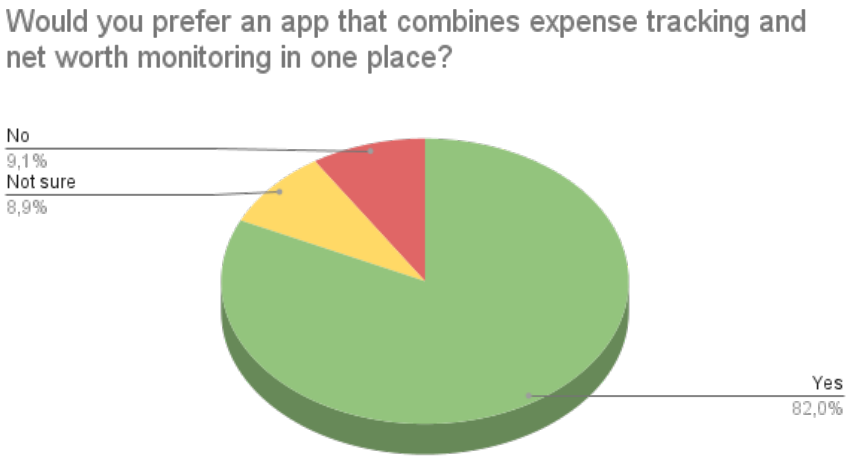

Figure 1.5: Preference for Combined Expense and Net Worth Tracking

Frustrations with current systems highlight critical usability gaps. Manual data entry, especially when users forget to input data (25.8%) or fail to gain meaningful insights (21.9%), was a dominant concern. A further 15.6% cited lack of bank support, and 9.3% expressed dissatisfaction across all areas (Figure 1.6).

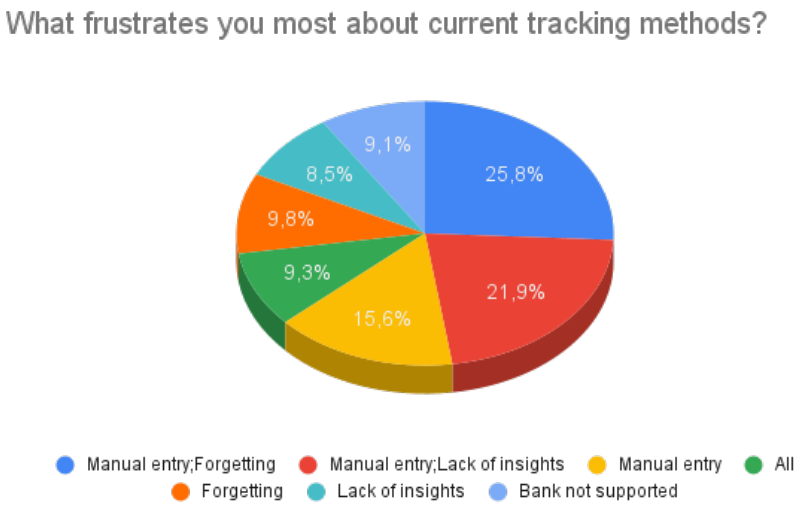

Figure 1.6: Main Frustrations with Current Tracking Methods

Regarding the most difficult expense types to track, shared expenses were the top concern (41.4%), followed by cash payments (31.0%) and subscriptions (19.3%). These categories often fall outside traditional digital capture systems, further complicating user workflows (Figure 1.7).

Figure 1.7: Expense Categories Users Find Hardest to Track

The role of PDF bank statements is also pivotal. Most users (71.8%) receive PDF statements either “often” or “always” (Figure 1.8), yet a majority still rely on manual

extraction or have no tools to automate the process. While 36.2% have tried manual extraction and 13.4% have used dedicated tools, 39.7% would like such tools but have not yet accessed them (Figure 1.9).

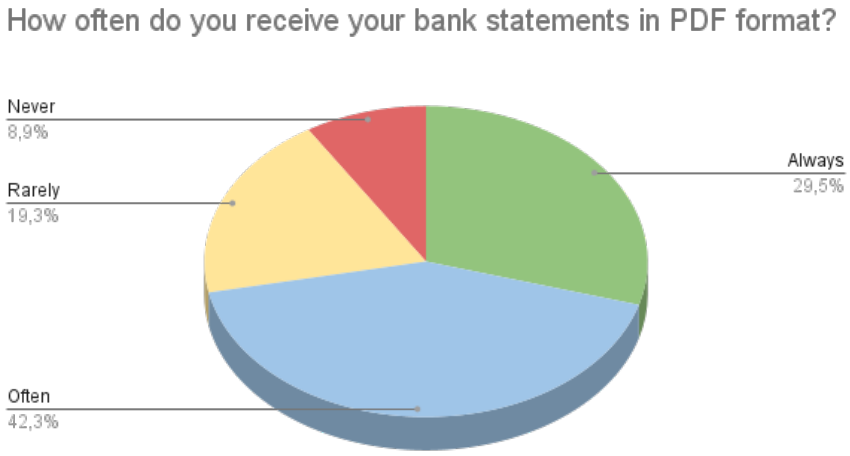

Figure 1.8: Frequency of Receiving PDF Bank Statements

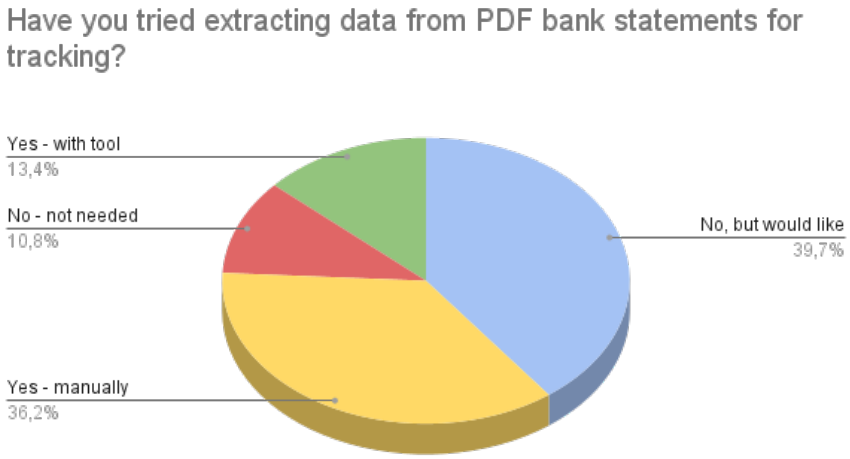

Figure 1.9: User Methods for Extracting Data from PDF Bank Statements

Finally, while skepticism toward AI remains, a majority of respondents express interest in AI-assisted financial advice, especially for general tips (26.7%) and insights based on spending behavior (13.7%). A growing segment (16.0%) even trusts AI for all types

of advice, reflecting a gradual shift toward acceptance of AI-driven recommendations (Figure 1.10).

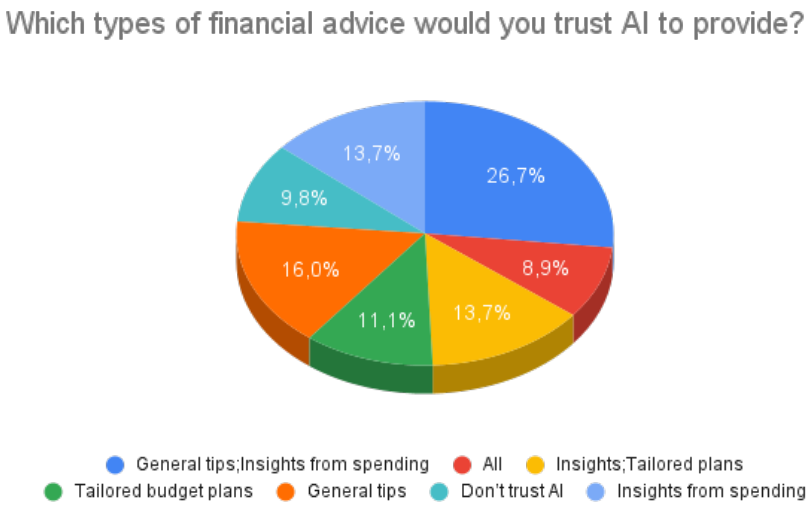

Figure 1.10: Types of Financial Advice Users Would Trust from AI Systems

1.4 Relevance of the Topic

The relevance of this research lies in its alignment with multiple current technological, societal, and academic trends. In an era where financial decisions are increasingly digitized, individuals are expected to manage complex financial activities such as budgeting, subscriptions, payments, and savings on their own. Yet, many people lack the time, knowledge, or tools to do so effectively. The consequences of poor financial awareness can be serious, from chronic overspending to long-term debt or missed financial opportunities. Against this backdrop, intelligent digital solutions that support personal finance management are becoming not only relevant but essential.

1.4.1 Technological Relevance

This thesis uses emerging technologies, including lightweight web frameworks (NestJS, React), document parsing tools (pdf-lib), and generative AI models (GPT-4o mini), to create a modern and modular financial assistant. The integration of a conversational AI into a secure, low-footprint web application demonstrates the feasibility of deploying advanced technologies outside of enterprise settings. Unlike complex AI solutions that require large-scale infrastructure or deep learning pipelines, fiTrack shows that even mini-

mal configurations can yield intelligent, interactive, and responsive systems. Furthermore, the project introduces a privacy-centric alternative to traditional financial applications by avoiding bank APIs and instead relying on document-based transaction extraction. This not only reduces dependencies on third-party institutions, but also enhances user trust by giving full control over their data. The technical implementation showcases how modular AI components can be combined to form useful tools without compromising performance or security - a valuable insight for developers and system designers in the AI and fintech fields.

1.4.2 Societal Relevance

At a societal level, financial literacy remains a critical challenge. Many individuals, especially younger generations, struggle to manage expenses, save money, and understand the implications of their financial behaviors. According to the OECD/INFE 2020 International Survey, young adults aged 18–29 consistently score lower in financial knowledge, attitude, and prudent behavior compared to older groups across most economies [16]. Tools that lower the barrier to entry for financial management, especially those that provide real-time, intuitive assistance, have the potential to significantly improve personal financial well-being. By designing fiTrack with accessibility and simplicity in mind, the project addresses a broader social need: empowering individuals to take control of their finances without requiring expert knowledge or financial background. In doing so, it contributes to efforts in closing the gap between financial literacy and technology adoption, particularly among younger, digitally native users.

1.4.3 Practical Relevance

From a practical point of view, the system provides a compelling solution to the problems faced by everyday users. Manual expense tracking remains time-consuming and error-prone. Although existing applications offer some level of automation, they often depend on sensitive data integrations or are limited in scope. The document-based approach taken in fiTrack removes the need for invasive bank connections and offers a faster, safer way to manage expenses based on files users already receive, such as monthly reports, receipts, or exports from digital wallets. Moreover, the inclusion of a conversational assistant powered by GPT-4o mini brings practical value by enabling users to interact with the system in a natural and flexible way. This goes beyond static dashboards or rule-based filters, offering instead dynamic summaries, guidance, and system explanations

in plain language. This functionality makes fiTrack a viable and appealing option for users with varying levels of digital or financial literacy. In addition, the work opens up potential directions for future research on hybrid AI systems, personalized financial recommendation models, and secure document-based workflows. The structured survey conducted as part of the evaluation also provides empirical data on user attitudes toward AI-driven financial tools, offering insights for both researchers and developers seeking to improve user experience in fintech applications.

1.5 Summary

The core problems identified through user research include heavy reliance on manual data entry, fragmentation across financial tools, limited support for local banking formats such as PDF statements, and an unmet demand for automation and intelligent assistance. These issues collectively highlight the need for an integrated platform that unifies expense tracking, net worth analysis, and AI-powered insights, while also supporting automation through technologies such as bank statement parsing and receipt scanning. *FiTrack* is designed specifically to address these problems by offering a localized, intelligent, and user-centric solution.

Chapter 2

FiTrack App

2.1 Aims of the Work

The primary aim of the FiTrack project is to develop an intelligent, unified web-based platform that addresses the key deficiencies observed in current personal finance applications. Unlike existing tools that typically focus on either expense tracking or net worth monitoring in isolation, FiTrack endeavors to offer a comprehensive financial management experience by integrating various aspects of personal finance into a single coherent system.

A significant objective of the platform is the reduction of manual data entry by automating the extraction of transaction data from bank statements, particularly from PDF documents issued by Kaspi Bank. In addition, FiTrack seeks to improve financial literacy by providing users with personalized insights and actionable recommendations, generated through the application of artificial intelligence to real transaction data.

Another important goal is to facilitate collaborative financial management for families or groups. By supporting shared accounts and incorporating a role-based access control (RBAC) mechanism, the system promotes transparency and efficiency in multi-user budgeting scenarios.

2.2 Novelty of the Project

FiTrack distinguishes itself from other personal finance applications through several innovative features designed to overcome common limitations in the market. Unlike typical solutions that emphasize either expense tracking or wealth monitoring, FiTrack

offers both functionalities within a unified interface. This allows users to develop a holistic understanding of their financial situation.

The system introduces automated extraction and categorization of transaction data from PDF bank statements - a capability that is rarely implemented, despite its utility. By enabling users to upload statements from Kaspi Bank and automatically process them, the application minimizes user effort and enhances data reliability.

Another novel aspect is the integration of an AI-powered chatbot capable of interpreting transaction histories and delivering customized financial advice, summaries, and recommendations. This feature elevates the platform beyond static dashboards or generic feedback mechanisms, offering intelligent, context-aware guidance tailored to individual user behavior.

Furthermore, FiTrack provides robust functionality for collaborative budgeting. It allows multiple users to manage shared financial goals while maintaining clearly defined roles and permissions. This feature, often overlooked or inadequately realized in other applications, supports practical financial planning in both family and group contexts.

Together, these innovations render FiTrack a more intelligent, comprehensive, and user-centric alternative to conventional financial tools.

2.3 Relevance of the Project

The increasing complexity of personal finance management underscores the relevance of the FiTrack project in the modern digital economy. Users frequently encounter fragmented tools that require switching between multiple applications or relying on manual data entry methods, such as spreadsheets. These approaches compromise efficiency and accuracy, impeding informed financial decision-making.

Moreover, automation in handling bank statements - particularly in PDF format and specific to local institutions like Kaspi Bank - remains underdeveloped. FiTrack directly addresses this gap by implementing a dedicated parsing mechanism tailored to such formats.

The scarcity of personalized, actionable financial recommendations based on actual transaction histories further highlights the need for solutions like FiTrack. By combining

AI-driven analytics with local banking realities, the platform delivers insights that are both contextually relevant and user-specific.

Consequently, FiTrack is poised to significantly enhance financial awareness, empower users to better manage their expenses and savings, and enable collaborative financial planning. These capabilities are particularly pertinent in an era of rapid digitalization, where efficient personal finance tools are increasingly vital.

2.4 Objectives

The development of FiTrack is guided by a set of well-defined objectives aimed at addressing real user needs while ensuring scalability, maintainability, and intelligent functionality. These objectives encompass the full project lifecycle - from initial research through design, implementation, and deployment.

The project commenced with comprehensive data collection and analysis, including user surveys and interviews. This empirical approach enabled the identification of pain points in personal finance management, such as the burden of manual transaction input and the lack of meaningful insights. The resulting user feedback informed the formulation of system requirements and shaped the design direction of the platform.

The architectural design of FiTrack emphasizes modularity and scalability. The system is logically divided into three core components: a backend, a frontend, and a microservice dedicated to parsing bank statements. This separation of concerns enhances maintainability, facilitates testing, and simplifies the integration of future enhancements.

Backend development, executed using NestJS and PostgreSQL, encapsulates the core business logic and operational capabilities. It manages key functions such as authentication, role-based access control, and the orchestration of interactions between various subsystems. Additionally, it handles the storage and processing of financial data, ensuring that application rules are enforced consistently across the platform.

The bank statement parsing service is implemented as a specialized microservice responsible for extracting structured transaction data from PDF statements, initially optimized for Kaspi Bank. This service significantly reduces manual labor for users and ensures that financial data is accurately categorized and normalized.

Frontend implementation, carried out using React and Material UI, focuses on delivering a clean, intuitive, and responsive user experience. Developed as a Progressive Web Application (PWA), FiTrack supports cross-platform accessibility, enabling seamless operation on both desktop browsers and mobile devices, including iOS [5].

The integration of an AI-powered chatbot via the GPT-4o mini API adds an intelligent layer to the user interface. The assistant interprets user spending behavior and delivers personalized financial feedback and summaries, providing meaningful, dynamic support beyond conventional static reporting.

Finally, all components of the system are deployed within a scalable hosting environment provided by ps.kz, with the application accessible via the public domain `https://fitrack.kz` [21]. This ensures high availability and prepares the platform for real-world use.

Together, these objectives establish the foundation for the methodology described in the subsequent section, which outlines the systematic process used to realize FiTrack and validate its core capabilities.

Chapter 3

Methods

The development of fiTrack is the result of a systematic approach that combines user needs analysis, modern technological solutions, and agile development practices. This methodology covers the entire product lifecycle - from requirements gathering and market analysis to MVP design, architecture implementation, and quality assurance. Each phase of the methodology is focused on creating a reliable, scalable, and user-friendly financial assistant that meets user expectations. The following sections provide a detailed overview of the methodology, starting with data collection and market analysis.

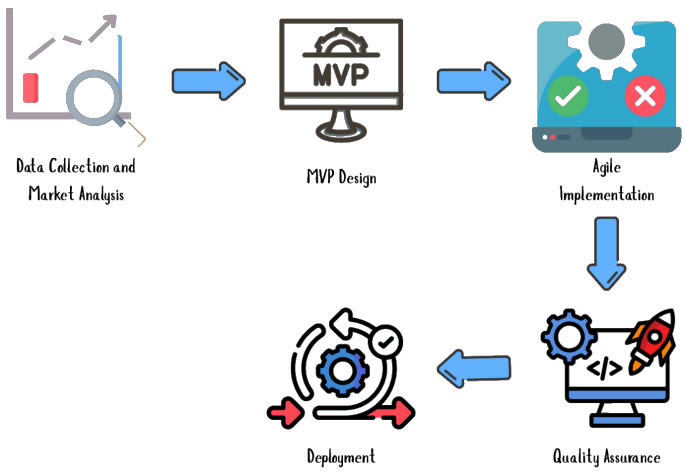

Figure 3.1: Overview of the Development Process
Icons from Flaticon.com

3.1 Data Collection and Market Analysis

To inform the design and feature set of the financial assistant application, a structured user survey was conducted with the primary goal of understanding real-world behaviors, pain points, and expectations related to personal finance management.

3.1.1 Objectives of the Survey

The survey was designed to identify current habits and tools users employ to manage personal finances, explore the challenges associated with both manual and semi-automated expense tracking, and evaluate interest and trust in AI-driven financial insights and automation. It also aimed to assess attitudes toward receipt scanning and automated expense entry, understand user concerns regarding data privacy and security in financial applications, and gather input on feature preferences that would drive long-term user engagement and retention.

These objectives were directly aligned with the core hypotheses of the product: that modern users struggle with manual financial tracking, are open to automation and AI if privacy is ensured, and expect intuitive, low-effort tools to assist them.

3.1.2 Survey Format and Method

The survey was distributed via Google Forms and included a mix of multiple-choice, Likert-scale, and open-ended questions. It was anonymous and required approximately 5–7 minutes to complete. The questionnaire was divided into six thematic sections:

The first section, Financial Habits, examined how users currently track their expenses, including whether they use apps, spreadsheets, or rely on memory and account balances. The second section, Challenges in Expense Tracking and Automation, identified specific difficulties users face when trying to track or analyze spending manually or semi-automatically, including common drop-off reasons. The third section, AI-Driven Insights and Financial Assistance, explored user interest in AI-generated reports, preferred formats such as chatbots or visual summaries, and the types of recommendations they find valuable. The fourth section, Receipt Scanning and Automated Expense Entry, assessed whether users currently retain receipts and how open they are to using automated scanning tools. The fifth section, Data Privacy and Security Concerns, captured user trust levels regarding financial data sharing, AI usage, and the types of security

they expect from financial tools. Finally, the sixth section, App Retention and Feature Preferences, collected feedback on previous app usage experiences, what led users to abandon them, and what features they would want in an ideal financial app.

3.1.3 Participant Profile

A total of 539 individuals participated in the survey, representing diverse backgrounds in terms of employment, income levels, and financial literacy. The sample size, while limited, provided qualitative and directional insights that were sufficient for hypothesis validation and initial product design decisions.

3.1.4 Justification of Approach

The decision to use a structured survey format allowed for the efficient collection of both quantitative and qualitative data at scale. The six thematic categories were intentionally designed to map directly to the anticipated feature areas of the application, enabling traceable connections between user needs and system design.

The full list of survey questions and response formats is provided in Appendix A.

3.2 MVP Design

Before any implementation began, we focused on designing the Minimum Viable Product (MVP) to validate core hypotheses and establish a scalable foundation. The architecture of the MVP follows a Service-Oriented Architecture (SOA) model, ensuring modularity, independent development, and simplified future scaling.

3.2.1 MVP Architecture

The system consists of three core services: the backend, the frontend, and the bank statement parsing service. Communication between these services is handled via RESTful APIs, enabling clear separation of concerns and flexibility in development.

Backend. The backend is responsible for business logic, data processing, and communication with the PostgreSQL database. It receives requests from the frontend—including authentication data, transactions, asset and liability details, or custom queries—processes

them, performs CRUD operations, and returns responses. It also manages user roles, session control, and security protocols.

Frontend. Built with React, Redux Toolkit, and Material UI, the frontend provides users with a responsive interface to manage their financial data [11, 13, 22]. It allows users to register, log in, input transactions, view statistics, categorize expenses, and receive financial recommendations. All data operations are handled via API calls to the backend.

Bank Statement Parsing Service. This service is designed to process PDF files containing bank transaction tables. It accepts input PDFs in a predefined format, extracts the data, and directly inserts it into the transaction table in the database. It does not serve data externally and only returns logs indicating success or failure. As a one-way service, it simplifies parsing workflows and minimizes integration complexity.

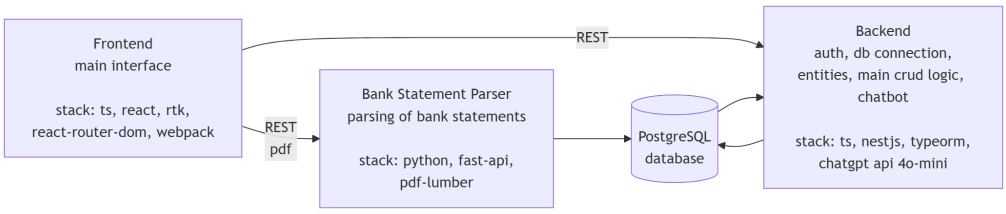

Figure 3.2: Service-Based MVP Architecture with Frontend, Backend, and Bank Statement Parsing Service

This modular architecture ensures each component can evolve independently and supports rapid iteration during MVP development.

3.2.2 Database Design

The backend utilizes PostgreSQL in combination with NestJS and TypeORM to define and manage the application's relational data models [15, 19, 24]. This technology stack was chosen for its balance between type safety, scalability, and efficient query performance.

The data model includes several essential entities that reflect both individual user data and their financial activities. These entities are summarized in Table 3.1.

Entity	Description
User	Contains personal details, login credentials, and user-specific configuration data.
Transaction	Stores all income and expense records, including amount, category, timestamp, and user linkage.
Asset	Represents user-owned financial and non-financial assets such as cash, savings, property, and investments.
Liability	Captures outstanding obligations like loans, mortgages, and credit card balances associated with a user.
Category	Defines custom and system-defined labels used to classify transactions for analytics and reporting.
Family	Enables users to create shared financial groups for managing household budgets and assets collectively.
Reset_Token	Handles secure password reset operations using time-bound tokens linked to individual users.

Table 3.1: Core entities in the FiTrack data schema and their functional purpose.

All entities are normalized and interconnected. For example, each transaction is linked to a user and a category, while users can belong to a family group. This structure supports data integrity and isolation between users, enables efficient data retrieval for statistics and analytics, and facilitates seamless integration with AI modules for classification, recommendation, and forecasting.

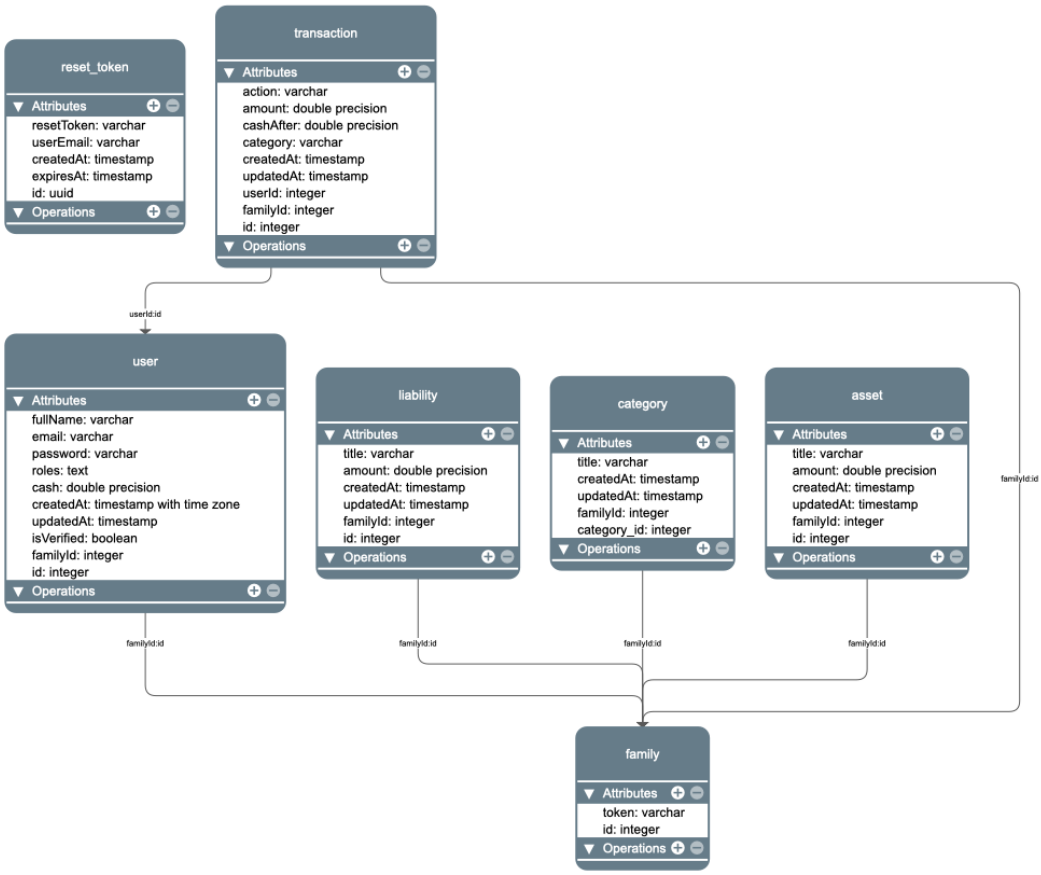

Figure 3.3: Entity-Relationship Diagram (ERD) for Core Financial Models

By focusing on clarity, modularity, and future extensibility, this MVP design provides a solid technical foundation for further development and testing of the financial assistant platform.

3.3 Agile Implementation

The project followed Agile development principles to deliver iterative functionality and maintain flexibility in response to feedback. Development was organized into short sprints with clearly defined goals, continuous integration, and regular testing cycles.

3.3.1 Backend

The backend of the application was developed using NestJS, a modular Node.js framework that supports enterprise-scale server-side applications. The primary responsibilities of

the backend include handling authentication, authorization, data persistence, business logic, and integration with external services such as the AI assistant module.

The backend codebase is organized according to modular architecture principles. Each feature domain (e.g., auth, user, transaction, chatbot) is encapsulated into its own module. A standard module typically includes:

Controller: Defines RESTful API endpoints and routes incoming HTTP requests to the appropriate service methods.

Service: Contains the business logic and orchestrates data processing, validation, and interactions with the database.

Entity (Model): Defines the database schema using TypeORM decorators.

DTOs (Data Transfer Objects): Specifies the shape and validation rules of incoming data.

Database Integration is implemented using TypeORM with PostgreSQL as the database engine. Repositories are used to abstract direct database operations, allowing for maintainable and testable code. Schema migrations ensure consistent and version-controlled database updates.

3.3.1.1 Authentication and Authorization

Authentication is based on JWT tokens, using the passport-jwt strategy. Each secured endpoint requires a Bearer token, which is decoded server-side to retrieve the user's identity. This authentication is centralized in the auth and user modules, which provide endpoints for:

```
POST /auth/register
POST /auth/login
POST /auth/reset-password
```

These features are implemented in accordance with the NestJS Passport guide.

Role-Based Access Control (RBAC) is enforced using strategy patterns. Two roles are supported:

admin: Full access to all resources.

member: Limited to creating transactions; read-only access to other resources.

The RBAC mechanism filters the data and blocks unauthorized actions at both the route and service layers.

3.3.1.2 API Structure and Swagger Documentation

The backend exposes RESTful endpoints for all core entities:

User: profile management

Transaction: financial entries

Asset & Liability: tracking financial positions

Category: user-defined transaction grouping

All endpoints support full CRUD functionality:

```
POST /[entity]
GET /[entity]
PATCH /[entity]/:id
DELETE /[entity]/:id
```

JWT tokens are mandatory for these endpoints. The backend decodes the token, applies role filters, and ensures data access is scoped to the authenticated user.

API Documentation is auto-generated using nestjs-swagger. All endpoints, query parameters, request bodies, and expected responses are available at:

```
https://fitrack.kz/api/docs
```

This documentation allows for direct testing and is fully synchronized with the current codebase.

3.3.1.3 Chatbot Service (AI Assistant Integration)

A dedicated chatbot module on the backend integrates GPT-4o mini through the LangChain framework. This module enables personalized financial analysis by dynamically incorporating user transaction data into AI prompts.

Workflow Description

The system accepts three input parameters: `useTransactions`, a boolean flag indicating whether transaction data should be used in the analysis; `period`, a string representing the time span of interest (allowed values are `'7'`, `'30'`, or `'90'` days); and `prompt`, a natural language query provided by the user.

If `useTransactions` is set to `true`, the system retrieves the user's transaction history from the database for the specified period. These transactions are then serialized into JSON format and appended to the user's original prompt.

The combined prompt is forwarded to the ChatGPT API via the LangChain framework, which orchestrates communication with the language model. The model processes the full prompt and generates a financial insight based on both the user's query and transaction data.

The final output is a natural language summary that provides the user with an AI-driven interpretation of their financial behavior.

Example API Interaction:

The client sends a POST request to the /chatbot/analyze endpoint, including an authorization token in the request header using the Bearer authentication scheme. The request body contains three fields:

useTransactions is set to true, indicating that the analysis should incorporate the user's past transactions.

period is set to "30", specifying that transactions from the past 30 days should be considered.

prompt contains the user's query, in this case: "Help me understand my spending

habits."

In response, the server returns a JSON object containing the field result, which holds a natural language summary generated by the AI. For this particular request, the response states:

"Over the past 30 days, your largest expenses were in Food (32

This module is optimized for asynchronous operation, allowing for latency-resilient performance and a responsive frontend experience.

Summary

The backend system is built with an emphasis on modularity, security, maintainability, and extensibility. By leveraging modern frameworks and patterns such as NestJS modules, RBAC strategies, DTO-based validation, and Swagger auto-documentation, it provides a robust foundation for the application. The chatbot integration further enhances the user experience by enabling real-time, AI-powered financial insights through cleanly abstracted service endpoints.

3.3.2 Frontend

The frontend of the application is built with React, using a component-based architecture designed for scalability and maintainability. It is responsible for rendering the user interface, managing client-side state, routing, and interacting with the backend API.

3.3.2.1 Routing and Access Control

React Router DOM is used for client-side routing. It defines public and protected routes, enabling navigation across the application. Protected routes are implemented using a custom `PrivateRoute` component that verifies user authentication and role permissions based on JWT tokens before rendering the target component.

3.3.2.2 Global State Management

The application uses Redux Toolkit for managing global state. It provides a clean structure through:

Feature-based slices (e.g., auth, transactions, categories),

Async thunks for side effects like API calls,

Custom middleware for token injection and centralized error handling.

State is kept normalized and scoped by feature. Loading states and error responses are tracked to ensure consistent UI feedback across the app.

3.3.2.3 API Communication

All API requests are handled via Axios. A global Axios instance is configured with request interceptors to automatically attach the `Authorization: Bearer <token>` header if the user is logged in:

This ensures secure and authenticated communication with the backend across all modules.

3.3.2.4 UI and Styling

The UI is built with Material UI (MUI) to ensure consistency, responsiveness, and accessibility. All components are styled using the MUI theme system, which enables centralized control of spacing, colors, typography, and breakpoints.

The layout is responsive and mobile-first, supporting both desktop and PWA-based mobile usage.

3.3.2.5 Build and Deployment

The application is bundled using Webpack. The build configuration is optimized for production with the following features:

Tree-shaking and JS/CSS minification,

Code splitting for route-level chunking,

PWA support with service worker generation (via Workbox).

The resulting single-page application is deployed to `https://fitrack.kz`, where it connects to a secured backend over HTTPS.

3.3.3 Bank Statement Parsing Service

A separate microservice was developed to automate the extraction of financial transactions from PDF bank statements. This service operates independently from the core application and is responsible for processing documents uploaded by the user.

3.3.3.1 Key Technologies

Technology	Description
Python	Selected for its ecosystem and strong support for data processing.
FastAPI	Used to expose the service as a lightweight and high-performance REST API.
pdfplumber	A library used to extract structured table data from PDF documents.
PostgreSQL	The target database where parsed transaction records are stored.

Table 3.2: Key technologies and their roles in the project.

3.3.3.2 Functionality Overview

The service accepts PDF files in a predefined format, typically containing tabular representations of transaction history (e.g., statements from Kaspi Bank). Upon receiving a file, the system:

Opens and scans the PDF using table-recognition logic.

Extracts relevant data from each row (e.g., transaction date, description, amount).

Transforms the data into a structured format consistent with the application's internal transaction model.

Inserts the data directly into the transactions table in the main database.

3.3.3.3 Response Logic

After processing, the service responds with the number of successfully inserted records and an execution status. In the case of errors (e.g., incorrect format, unreadable content),

a descriptive error message is returned.

3.3.3.4 Integration

This microservice is integrated into the broader backend system via internal HTTP communication. The main application relies on it to enrich user accounts with imported financial data from scanned PDF statements.

3.3.3.5 Visualization

To aid understanding, a sample PDF page and a corresponding database table screenshot can be included in the appendix or supplementary documentation. This illustrates how raw tabular data is extracted and persisted.

3.4 Quality Assurance

Testing was tightly integrated into the development cycle and conducted iteratively after the implementation of each feature or module. The primary goal was to ensure not only the correctness of functionality but also the robustness, security, and usability of the system as it evolved.

Backend services such as authentication, transaction handling, and AI analysis were verified through manual and semi-automated functional testing. After implementing business logic in each module, data flow correctness and expected behavior were manually validated by directly interacting with the exposed endpoints. Tools like Postman [20] and Swagger UI [23] played a critical role in testing API behavior, verifying request structures, authentication mechanisms, and expected responses under various input conditions.

Although formal unit testing frameworks were not employed, each feature was tested immediately after development to catch logical and structural issues early. This approach allowed for rapid feedback and reduced the likelihood of cascading bugs in later stages.

After each sprint, regression testing was conducted manually to confirm that newly introduced functionality did not break or degrade the behavior of previously tested features. Special attention was given to features involving user roles and permissions to ensure strict enforcement of RBAC policies and secure access boundaries.

Front-end validation included testing navigation between public and private views, Redux state updates, and interaction flows such as form inputs and submission handling. Although no automated UI test suite was used, real-time interaction testing in the browser ensured a consistent and intuitive user experience.

Overall, the testing process emphasized stability, iteration, and full-stack traceability of issues, aligning closely with the Agile methodology used in the project.

3.5 Deployment

The application was deployed on a VPS server provided by ps.kz, with all services running on the same machine but on different ports:

Frontend on port 3000

Backend on port 3003

Bank statement parsing service on port 8000

The frontend was built as a Progressive Web App (PWA) using **npm run build**, which compiled and optimized the React source code into production-ready static assets. These assets were served through a Node.js server. Additional optimizations such as gzip compression were applied to improve load times and reduce bandwidth usage.

For the iOS platform, the frontend was converted into a native-like mobile application using Capacitor [8] and Xcode [1], enabling installation and usage on Apple devices while maintaining PWA advantages.

The deployment pipeline involved the use of Git [3] for version control, with three separate repositories maintained for the frontend, backend, and bank statement parsing service respectively. Updates were deployed by pulling the latest code from these repositories, running the build process (for frontend), and restarting services manually via SSH.

The application was hosted under the domain fitrack (https://fitrack.kz/), with HTTPS enabled using a Let's Encrypt SSL certificate [6] configured through ps.kz's hosting panel, ensuring secure communication between clients and the server.

Only a single environment was maintained, which served as both the development and production environment, streamlining the deployment and testing process.

Process management and monitoring were handled using PM2 [9], which facilitated running Node.js services persistently, managing logs, and enabling easy restarts or reloads of applications after deployment.

3.6 Summary

The development process, from initial user research and MVP design to modular implementation, testing, and deployment, was guided by clear architectural decisions and user-centric priorities. Each stage of this methodology laid the foundation for functional features, security mechanisms, and intelligent automation. With the application successfully deployed and operational under production conditions, we now turn to an evaluation of the realized features.

Chapter 4

Results

This chapter presents the outcomes of the FiTrack application development and aligns them explicitly with the methodology outlined in Chapter 3. The features described here were implemented based on the architectural plans (Section 3.2.1), backend and frontend frameworks (Sections 3.3.1 and 3.3.2), and the bank statement parsing microservice (Section 3.3.3). The results also reflect core user requirements and survey insights gathered during the Data Collection phase (Section 3.1).

4.1 Dashboard

The Dashboard functions as the central user interface, providing a real-time overview of financial activity. In accordance with the MVP architecture (Section 3.2.1), it integrates both manual and automated transaction input methods. Manual inputs allow users to record income and expenses with attributes such as amount, category, date, and description. Automated input is facilitated through the PDF upload functionality, which invokes the bank statement parsing microservice (Section 3.3.3), enabling streamlined ingestion and categorization of transaction data. The Dashboard also includes visualizations such as a donut chart for expense distribution and a transaction journal with search and filter capabilities, features directly informed by survey feedback (Section 3.1).

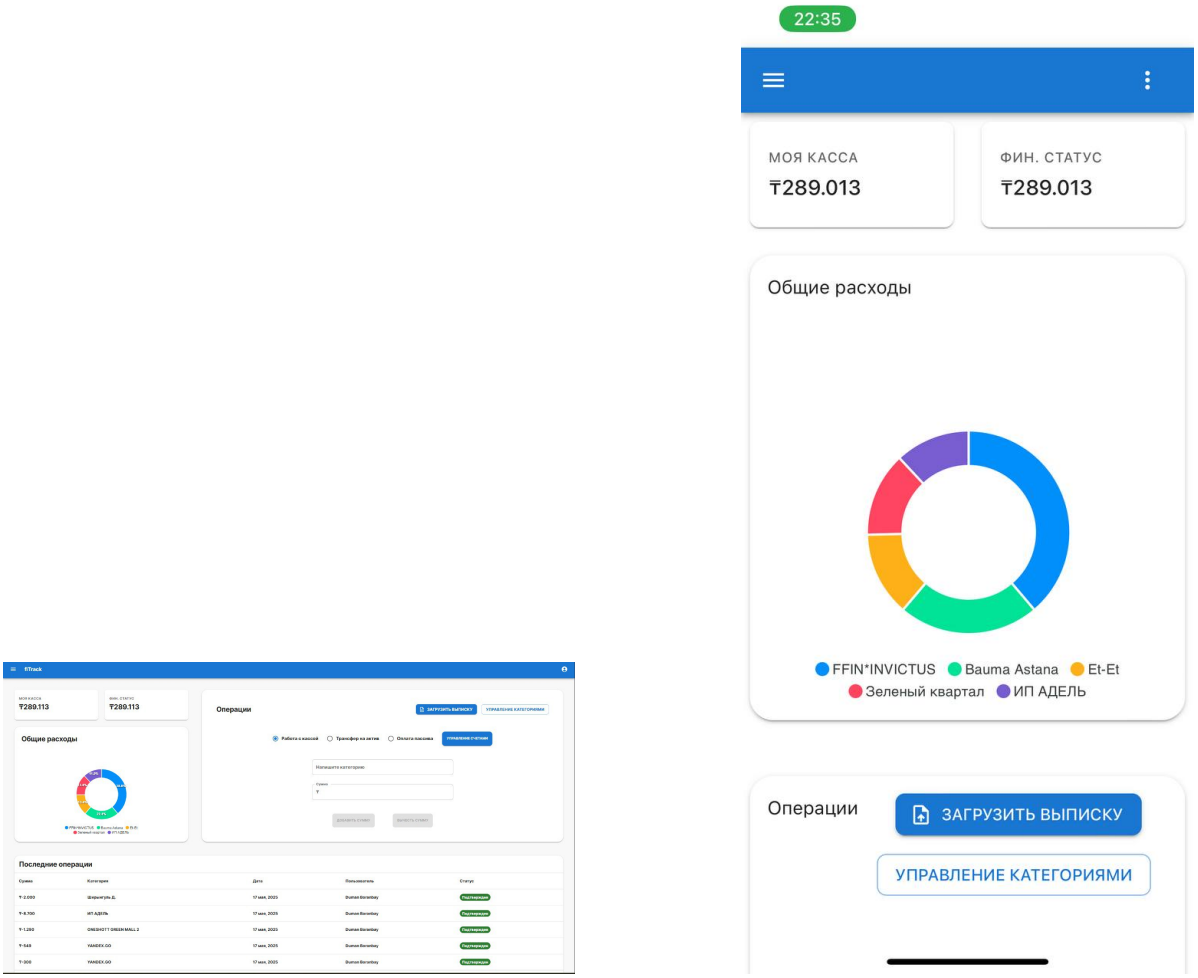

Figure 4.1: Dashboard on web (left) and mobile (right).

4.2 Asset and Liability Management

Asset and Liability Management Reflecting the data model defined in Section 3.2.2, this section of the application enables users to track assets (e.g., property, investments) and liabilities (e.g., loans, credit). The interface dynamically computes and displays net worth by subtracting liabilities from assets in real time. This component fulfills the identified user need for holistic financial visibility and is tightly integrated with the database schema introduced in the MVP design.

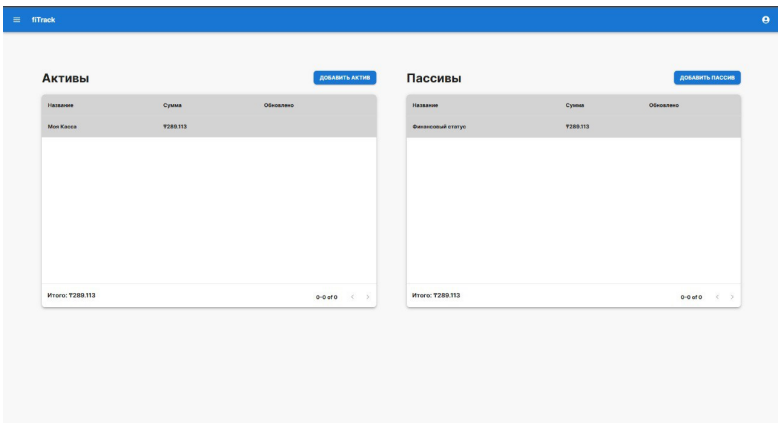

Figure 4.2: Asset and Liability Management page on the web version.

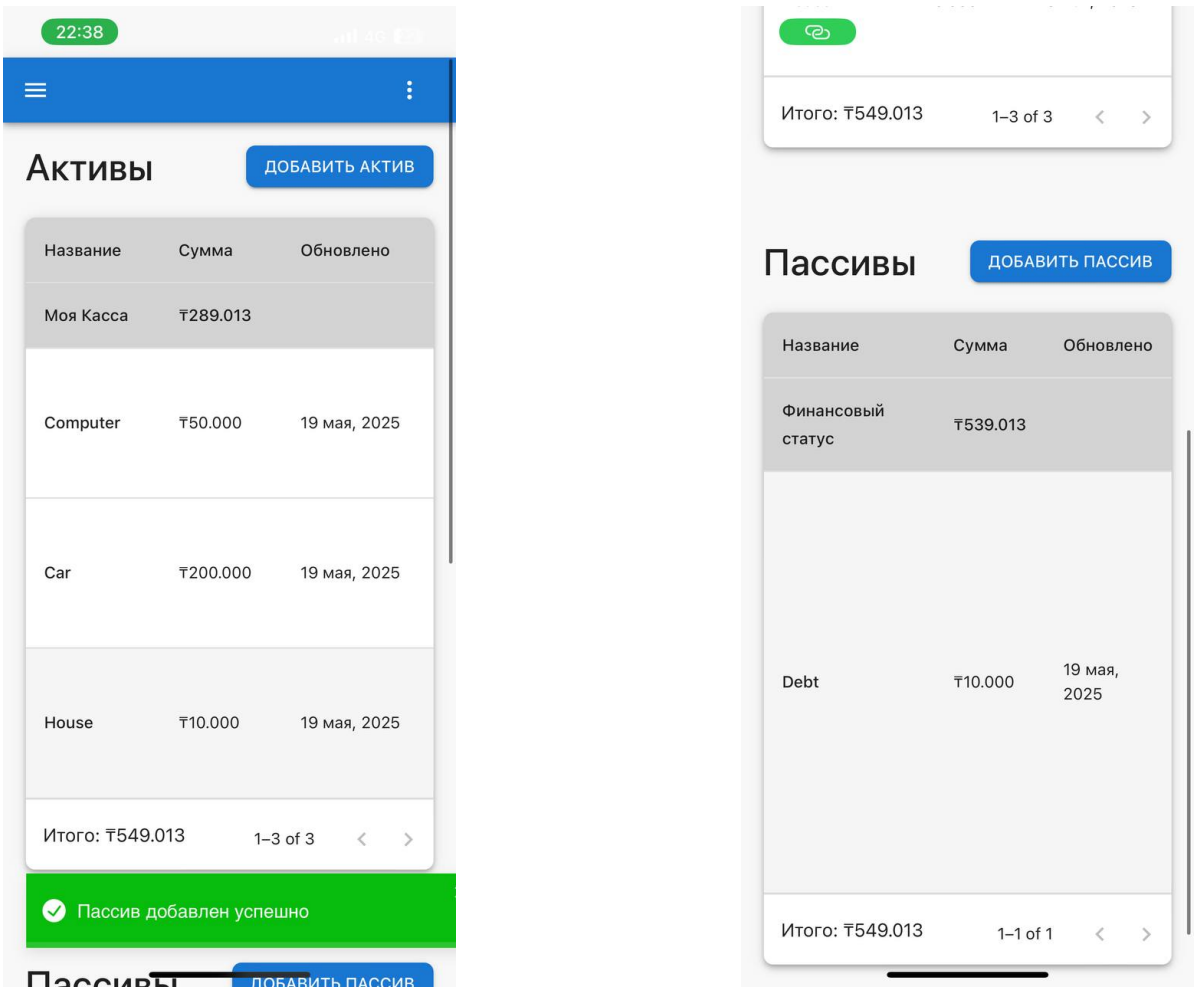

Figure 4.3: Mobile view: Asset table (left) and Liability table (right).

4.3 Chatbot Assistant

The AI-powered assistant, developed as described in Section 3.3.1.3, offers personalized financial analysis using natural language inputs. It leverages the LangChain-integrated GPT-4o mini model and backend logic to process recent transactions and deliver insights. The system currently supports analysis over the last 7, 30, or 90 days of the transaction history, allowing users to specify the desired period for financial insights. This feature aligns with strong user interest in AI-driven support, as highlighted in the survey (Section 3.1). The assistant's ability to handle prompts like "What did I spend most on last month?" or "Where can I save?" is supported by the chatbot module's query structure and context-aware logic (Section 3.3.1).

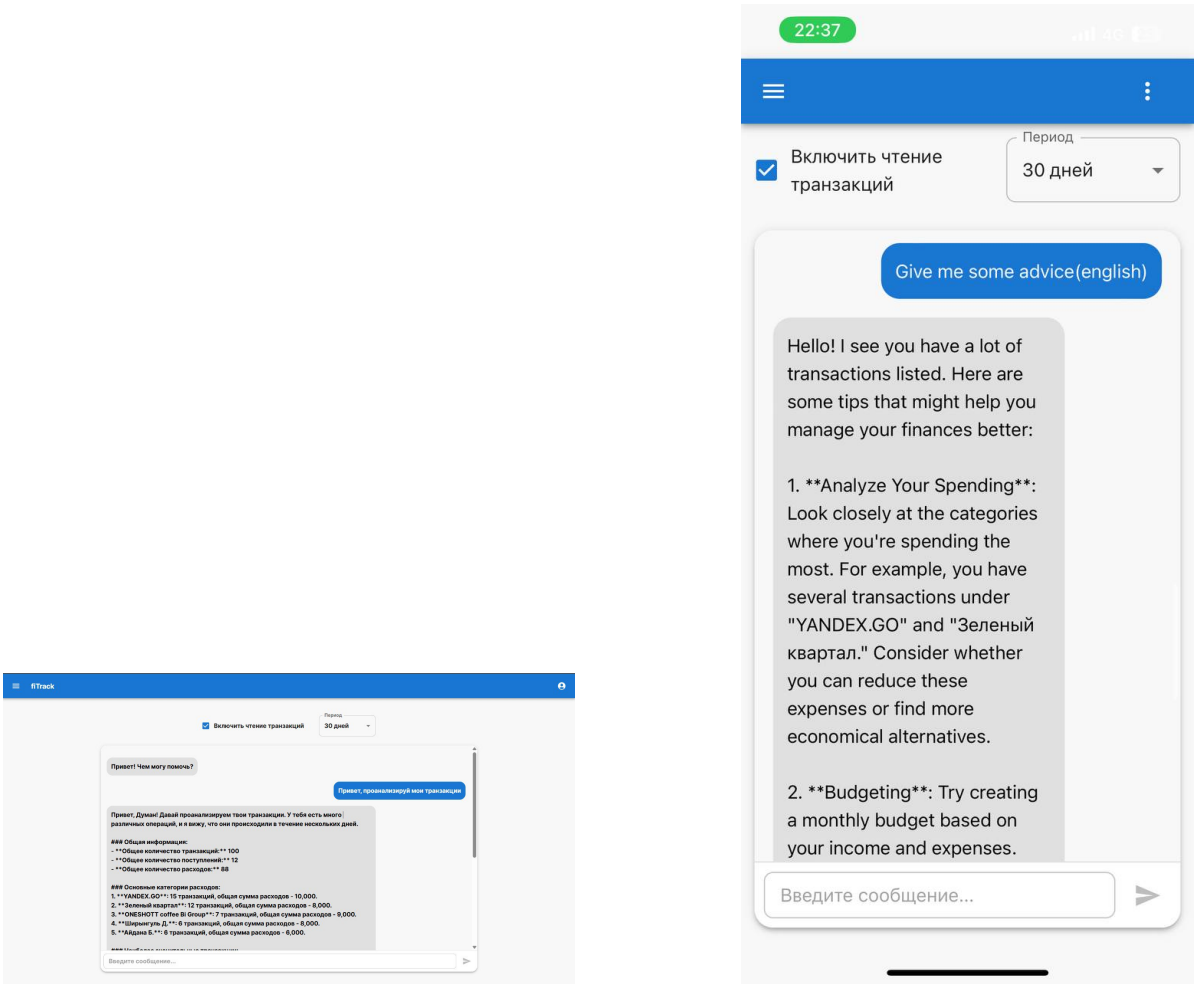

Figure 4.4: Chatbot interface on web (left) and mobile (right).

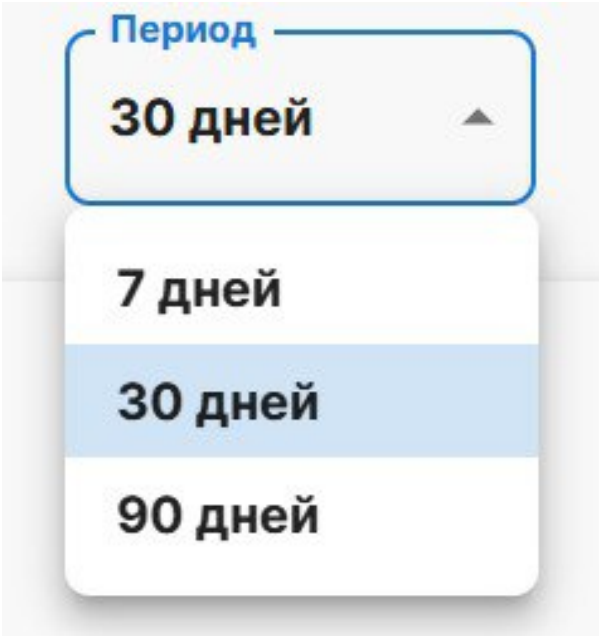

Figure 4.5: Period selection feature in the chatbot interface, allowing users to analyze transactions over the last 7, 30, or 90 days.

4.4 Bank Statement Parsing

This screenshot shows the user interface for uploading a PDF bank statement in the FiTrack application. The user has selected a PDF file through the standard Windows file picker. Upon clicking the "Upload Statement" button, the file is sent to the backend, where a dedicated microservice processes it by extracting transactions from the table inside the PDF and saving the data directly to the database. This workflow is implemented as part of the bank statement parsing service described in Section 3.3.3 Bank Statement Parsing Service of the methodology.

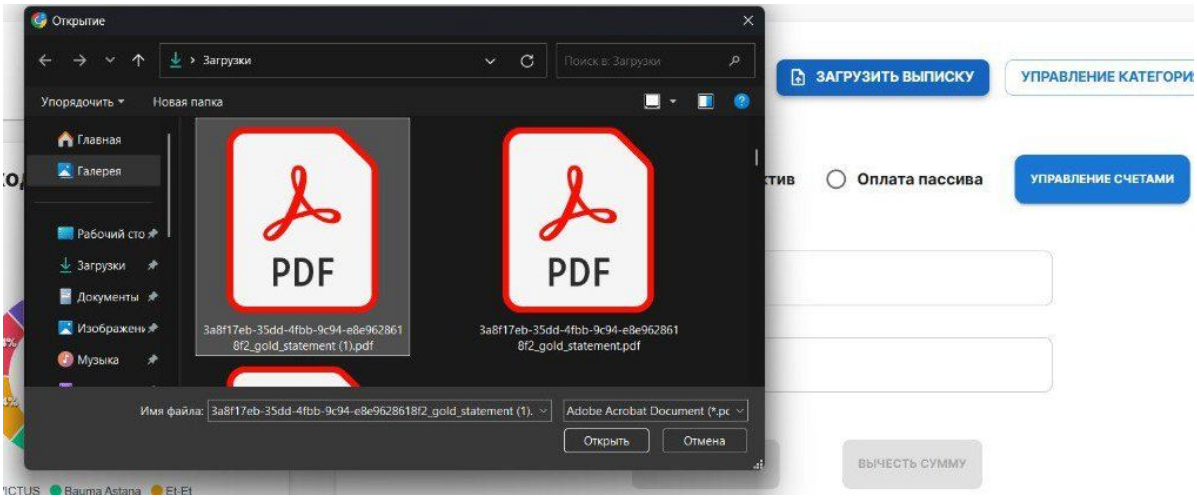

Figure 4.6: User interface for uploading a PDF bank statement in the FiTrack application.

4.5 User Authentication: Login and Sign Up

User Authentication: Login and Sign Up Built according to the security design described in Section 3.3.1, the authentication system allows for secure registration and login using JWT tokens. This implementation supports both user roles and access control, as dictated by the RBAC system. The interface follows frontend design principles laid out in Section 3.3.2, ensuring a responsive experience across devices.

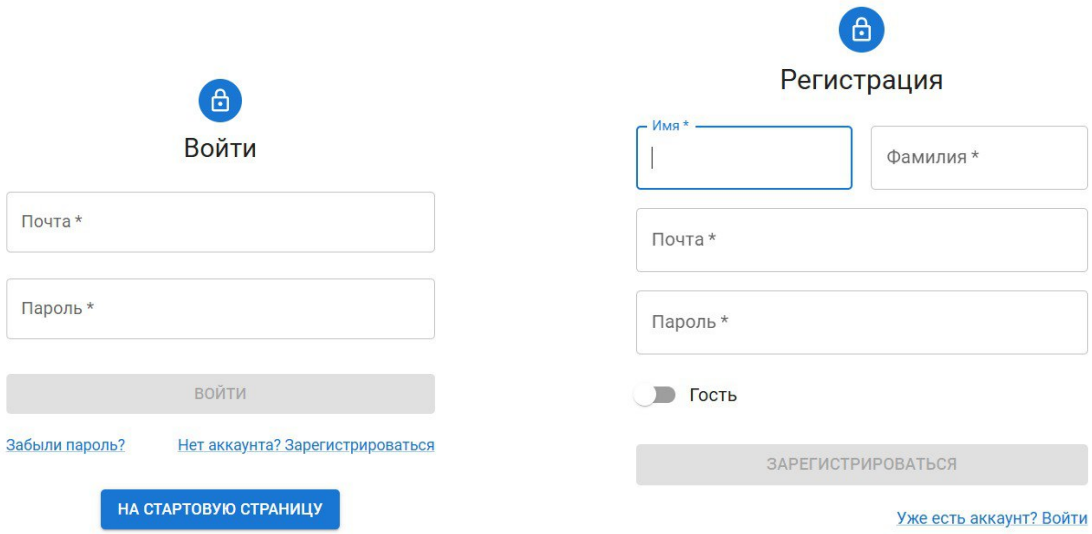

Figure 4.7: Web version: Login page (left) and Sign Up page (right).

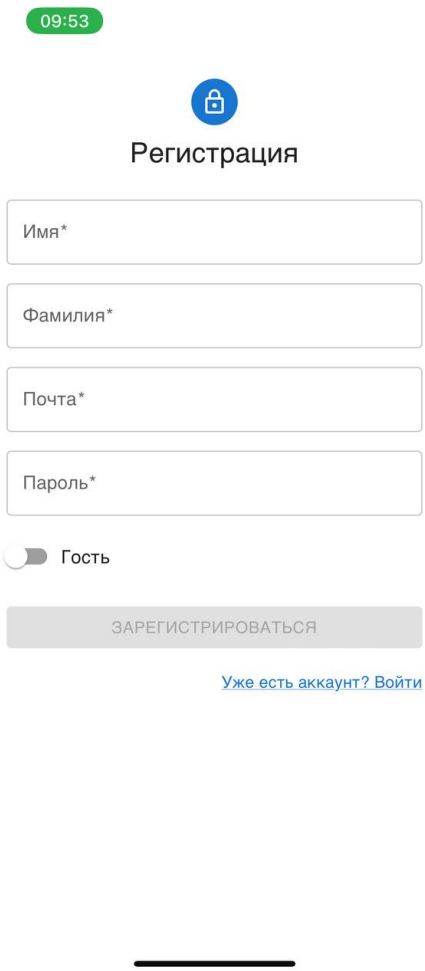

Figure 4.8: Mobile version: Sign Up page.

4.6 Profile Management

Profile Management The profile page, designed using frontend methodologies (Section 3.3.2), enables users to manage personal data, update credentials, and control access. This contributes to the usability and personalization priorities identified in the initial user research.

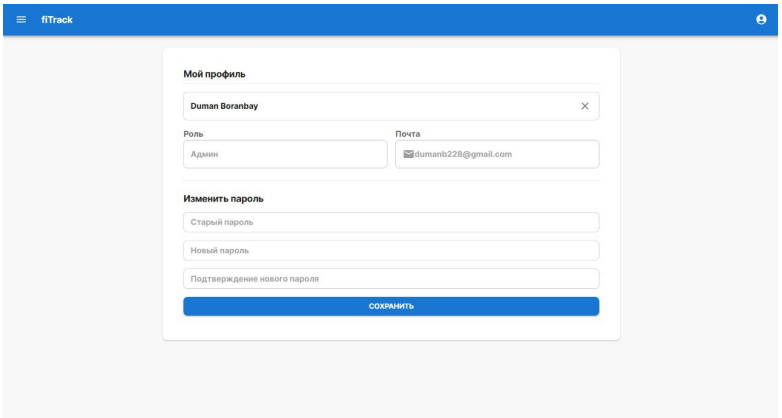

Figure 4.9: Web version: Profile management page with options to edit user data and change password.

4.7 API Documentation Overview

API Documentation As specified in Section 3.3.1, the application exposes its functionality through a fully documented Swagger-based REST API. Users and developers can explore, test, and integrate with the backend via interactive documentation, reflecting the backend's modular structure and adherence to best practices in maintainability and transparency.

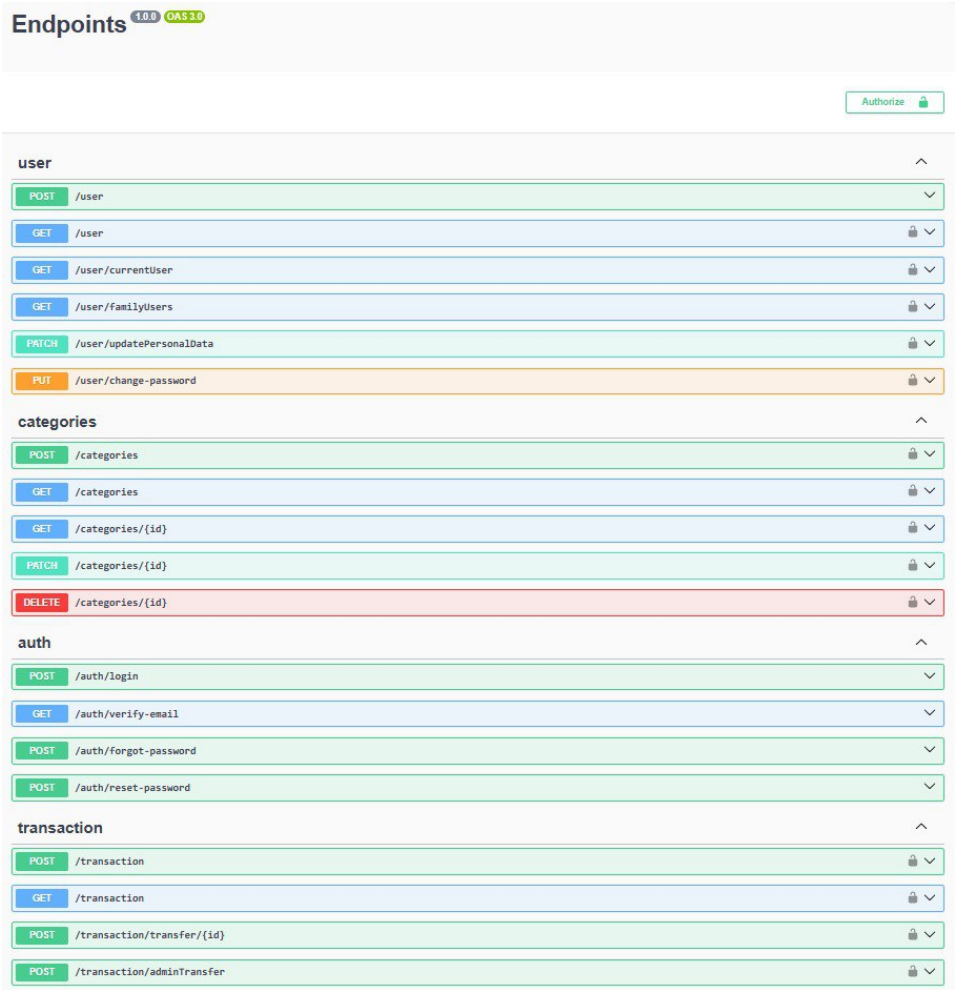

Figure 4.10: Full view of the Swagger API documentation.

Entity	Description
Asset	Endpoints for creating, reading, updating, and deleting personal assets.
Liability	Manage personal liabilities such as debts and loans.
Transaction	Record and retrieve transactions with filtering options.
Auth	Handles login, registration, and token management.
Reset_Token	Manages password reset tokens with validation and expiration.
User	View and update user profiles and roles.
Family	Create and manage family groups and shared finance.
Category	Create, update, and delete transaction categories.
Chatbot	Submit user queries, retrieve responses, and manage chatbot settings.

Table 4.1: Overview of available API endpoints and their functionality.

4.8 Summary

Each implemented feature directly corresponds to components described in the methodology. The Dashboard and PDF import validate the design of the bank statement parsing service. Asset and liability tracking realize the database schema. The chatbot assistant proves the feasibility of transaction-based AI interaction. Authentication and profile management support the secure and user-friendly interface. Together, these results confirm the coherence between the planned architecture, implementation process, and user-facing outcomes. The consistency between the envisioned system architecture and its realized components provides a strong foundation for further reflection. The next chapter discusses the implications of these results, examines remaining limitations, and outlines potential directions for future enhancements.

Chapter 5

Discussion

The results of the evaluation (as detailed in Chapter 4) demonstrate that FiTrack provides a meaningful enhancement to personal financial management by combining AI-driven automation with real-time analytics. Users consistently reported that features such as automatic transaction extraction and personalized chatbot recommendations significantly reduced manual effort and increased the usability of the platform. The dashboard interface, coupled with the conversational assistant, was noted for improving both user engagement and financial awareness. These findings support the hypothesis that the integration of artificial intelligence into everyday financial tools can streamline core workflows, reduce cognitive load, and improve the overall decision-making process for users. By automating repetitive tasks such as data entry and categorization, FiTrack allows users to focus more on interpreting their finances and planning ahead, rather than on managing the mechanics of data organization.

5.1 Limitations

Despite the promising results, several limitations were identified in the current implementation of FiTrack. These are primarily a result of constraints imposed by the MVP (Minimum Viable Product) stage and existing technological or financial factors. One notable limitation is the requirement for manual document input. FiTrack does not currently support integration with bank APIs, meaning users must upload their bank statements in PDF format manually. This adds friction to the user experience and limits the platform's ability to provide real-time synchronization with financial accounts. Another challenge stems from the dependency on the structural consistency of PDF docu-

ments. The transaction extraction algorithm relies heavily on the predictable formatting of bank statements. Any variations or changes in layout by different banks can result in parsing errors, which require updates to the extraction logic and reduce the robustness of the system. The current infrastructure of the platform is also constrained by limited server capacity. Since FiTrack was deployed on a budget-limited VPS during its MVP phase, it may experience performance degradation under high load. This is especially evident when several users interact with the AI assistant or upload large documents simultaneously. Furthermore, there are limitations associated with the conversational AI assistant. The GPT-4o mini model used in the chatbot has a fixed context window, restricting the amount of transactional data that can be analyzed in a single request. To stay within the model's input limits, the system currently supports analysis over the last 7, 30, or 90 days of the transaction history only (as shown in Figure 4.5). This hampers the chatbot's ability to provide insights based on long-term financial trends. Lastly, the cost structure of the AI component presents its own limitations. Since usage of large language models is billed per token, frequent or complex chatbot interactions quickly accumulate costs. To remain within budget, FiTrack currently restricts the depth and frequency of queries, which may affect the responsiveness and flexibility of the assistant for heavy users.

5.2 Future Directions

Several promising directions for further development emerged based on user feedback and internal technical analysis. First, expanding support for more banks and regional formats will be essential for improving accessibility. Enhancing the document parser to handle a wider range of PDF templates will reduce onboarding friction and make the platform more inclusive for users in different financial systems. Another key area is the improvement of the chatbot's personalization capabilities. By introducing adaptive feedback loops, behavioral segmentation, and more precise user profiling, the assistant could deliver more contextually relevant and accurate financial recommendations tailored to individual spending patterns. In addition, FiTrack could benefit from the integration of advanced analytics and forecasting mechanisms. By incorporating trend detection, predictive budgeting, and proactive alerts, the platform would empower users to make more strategic financial decisions, not just reactive ones. Visualization and reporting also offer room for refinement. Developing more interactive and dynamic dashboards—featuring comparative analysis, trend visualizations, and customizable KPI tracking—could enhance the interpretability of

financial data and promote ongoing user engagement. Looking further ahead, integrating with banks and financial institutions via secure APIs would eliminate the need for manual statement uploads entirely. Such a step would not only improve user convenience but also enable real-time data synchronization, a critical component for a truly modern financial assistant. Finally, upgrading the system's infrastructure to a scalable cloud-based architecture with usage-based billing would address current performance bottlenecks and provide the flexibility needed to support a growing user base. This would also allow the AI assistant to scale its capabilities more dynamically in response to varying loads and demands.

Chapter 6

Conclusion

This project resulted in the development and implementation of FiTrack, a personal finance application specifically designed to address key limitations in existing financial management tools. FiTrack's primary focus was to reduce manual efforts in transaction entry, integrate both expense tracking and asset management, and provide meaningful, user-adaptive insights through automation and AI.

6.1 Key Achievements

Automated Transaction Processing. One of the core achievements was the development of a system capable of automatically extracting and categorizing transaction data from PDF bank statements. This significantly reduced the need for manual input, improved accuracy, and simplified day-to-day financial tracking.

Personalized AI-Driven Analytics. By integrating a GPT-4o mini-powered chatbot, the platform delivers dynamic and personalized feedback based on user transactions. This transforms raw financial data into actionable insights, tailored to individual behavior and context.

Unified Expense and Net Worth Management. Unlike conventional tools that separate budget tracking from long-term wealth management, FiTrack offers a comprehensive view of the user's financial landscape—covering income, expenses, assets, and liabilities in a single interface.

User Validation. Evaluation through structured surveys and usage feedback confirmed

that FiTrack effectively reduces friction, increases clarity, and enhances user awareness—especially among those with limited financial literacy or prior exposure to such tools.

6.2 Practical Implications

The project demonstrates that combining automation, personalization, and integration into a unified financial assistant can address real-world pain points faced by everyday users. By eliminating redundant tasks like manual transaction entry and offering AI-generated recommendations, FiTrack helps users develop better spending habits and gain a clearer understanding of their financial situation. Its modular, user-friendly design and support for locally relevant bank formats (e.g., PDF statements from Kaspi Bank, Figure4.6) make the platform both accessible and adaptable. The ability to serve users with varying levels of technical and financial knowledge further highlights its real-world applicability.

6.3 Final Remarks

FiTrack was developed in direct response to clearly identified user challenges-manual input, fragmented visibility, and lack of personalized insight. It successfully combines the core functions of modern personal finance tools into one cohesive, intelligent solution. As demonstrated in the early results, the system lays the groundwork for a smarter, more efficient future in personal finance management.

By addressing key gaps highlighted in both academic literature and user feedback-namely, the need for automation, holistic financial tracking, and context-aware guidance-FiTrack not only validates its original hypotheses but also proposes a scalable model for the next generation of financial applications.

This work sets a reference point for how artificial intelligence and thoughtful design can converge to support real, everyday financial decision-making. Continued refinement, particularly in areas like real-time bank integration and AI scalability, will further enhance its impact and relevance in an increasingly digital and data-driven economy.

Appendix A

User Survey: Financial Management and Expense Tracking

Objective: To identify real user pain points, habits, and challenges in financial management to develop relevant and effective features.

A. Personal Financial Overview

1. What aspects of your personal finances do you currently track?

   ☐ Expenses

   ☐ Income

   ☐ Net worth (assets and liabilities)

   ☐ I don't track my finances

2. Would you prefer an app that combines expense tracking and net worth monitoring in one place?

   ☐ Yes

   ☐ No

   ☐ Not sure

B. Manual Input & Automation Challenges

3. How do you currently track your expenses?

   ☐ I do not track

   ☐ Manually in a spreadsheet

   ☐ In an app (manual entry)

   ☐ In an app (automated bank connection)

4. What frustrates you most about current tracking methods? (Select all that apply)

   ☐ Time-consuming manual entry

   ☐ Forgetting to update

   ☐ Lack of insights or analysis

   ☐ Apps don't support my bank

5. How often do you receive your bank statements in PDF format?

   ☐ Always

   ☐ Often

   ☐ Rarely

   ☐ Never

6. Have you tried extracting data from PDF bank statements for tracking?

   ☐ Yes – manually

   ☐ Yes – with a tool

   ☐ No, but I would like to

   ☐ No – not needed

C. Transaction Review & Insight Usefulness

7. What types of expenses are hardest for you to keep track of?

   ☐ Cash payments

   ☐ Subscriptions

   ☐ Shared/family expenses

   ☐ Other: _____

8. How useful would the following insights be for you? (1 = Not Useful, 5 = Very Useful)

Insight Type	1	2	3	4	5
Top spending categories	○	○	○	○	○
Alerts on excessive spending	○	○	○	○	○
Budget predictions based on past habits	○	○	○	○	○
Net worth progress over time	○	○	○	○	○

D. AI Advice & Personalization

9. Which types of financial advice would you trust AI to provide?

   ☐ General tips to save money

   ☐ Insights from my own spending habits

   ☐ Budget plans tailored to my goals

   ☐ I don't trust AI for advice

10. How would you prefer to receive AI-generated financial advice?

    ☐ Real-time alerts

    ☐ Weekly email reports

    ☐ Chatbot interaction

    ☐ Visual dashboards

- ☐ None

11. If AI could answer one question about your finances, what would it be? _____

E. Shared Budgeting / Collaboration

12. Do you manage finances with anyone else (e.g., partner, family)?
    - ☐ Yes
    - ☐ No
13. Would you find it helpful to track shared budgets with others in the same app?
    - ☐ Yes
    - ☐ Maybe
    - ☐ No

F. Receipt Management & Automation

14. What do you typically do with your receipts?
    - ☐ Discard them
    - ☐ Keep physical copies
    - ☐ Photograph or scan
    - ☐ Enter manually in app
15. Would you find automatic receipt scanning helpful?
    - ☐ Yes – to save time
    - ☐ Yes – if very accurate
    - ☐ No – prefer manual entry

16. How important are the following receipt details? (1 = Not Important, 5 = Very Important)

Detail	1	2	3	4	5
Store name and location	○	○	○	○	○
Itemized list	○	○	○	○	○
Expense category	○	○	○	○	○

G. Security & Privacy

17. Are you concerned about storing your financial data in an app?

    ☐ Yes – data breaches

    ☐ Yes – don't trust AI

    ☐ No – I'm comfortable

    ☐ It depends on app security

18. What level of data protection do you expect?

    ☐ Local-only storage

    ☐ End-to-end encryption

    ☐ Cloud sync with strong security

    ☐ Other: _____

19. Would you trust AI if all processing happens locally and is encrypted?

    ☐ Yes

    ☐ Yes, with conditions

    ☐ No

H. App Features & Retention

20. Which financial apps have you used before? Why did you stop? _____

21. What features would your ideal personal finance app include? _____

22. How useful would these features be to you? (1 = Not Useful, 5 = Very Useful)

Feature	1	2	3	4	5
Voice input for transactions	○	○	○	○	○
Auto-categorization of expenses	○	○	○	○	○
Shared/family budgeting	○	○	○	○	○
Visual dashboard of finances	○	○	○	○	○

23. Which task would you most like to automate?

    ☐ Subscription management

    ☐ Report generation

    ☐ Smart budget limits

    ☐ Bank statement parsing

Additional Resources

For detailed API documentation related to this project, see the Swagger API docs available at:
`https://skitter-mustang-f75.notion.site/For-API-documentation-section-1f8de82b12f4801c8`

Bibliography

[1] Apple Inc.
Xcode, 2025.
URL https://developer.apple.com/xcode/.

[2] Empower.
Empower personal dashboard: Track your investments, net worth, and retirement.
https://www.empower.com, 2024.
Accessed: 2025-05-23.

[3] Git Community.
Git Version Control System, 2025.
URL https://git-scm.com/.

[4] Google DeepMind.
Gemini: Google's multimodal ai models.
https://deepmind.google/technologies/gemini, 2023.

[5] Google Developers.
Progressive web apps (pwa) overview, 2022.
URL https://web.dev/progressive-web-apps/.

[6] Internet Security Research Group.
Let's encrypt, 2025.
URL https://letsencrypt.org/.

[7] Intuit Inc.
Mint budgeting app review, 2024.
URL https://moneywise.com/managing-money/budgeting/mint-com.

[8] Ionic Framework.

Capacitor documentation, 2025.
URL https://capacitorjs.com/docs.

[9] Keymetrics.
PM2 - Node.js Process Manager, 2025.
URL https://pm2.keymetrics.io/.

[10] Medius.
How ai is transforming expense management, 2023.
URL https://www.medius.com/resources/ai-expense-management/.

[11] Meta Platforms, Inc.
React Documentation, 2024.
URL https://react.dev/.

[12] Moneywise.
Mint vs. empower vs. ynab.
Online Article, 2024.
URL https://moneywise.com/managing-money/budgeting/mint-vs-empower-vs-ynab.

[13] MUI Team.
Material UI, 2024.
URL https://mui.com.

[14] NerdWallet.
Best budgeting apps of 2025.
Online Review, 2025.
URL https://www.nerdwallet.com/article/finance/best-budget-apps.

[15] NestJS Team.
NestJS: A Progressive Node.js Framework, 2023.
URL https://nestjs.com/.

[16] OECD.
Oecd/infe 2020 international survey of adult financial literacy, 2020.
URL https://www.oecd.org/en/publications/oecd-infe-2020-international-survey-of-adu
145f5607-en.html.

[17] OpenAI.
GPT-4o API Documentation, 2024.
URL https://platform.openai.com/docs/models/gpt-4o.

[18] Phroogal.
Spendee app review: Pros cons and best budget features, 2025.
URL https://www.phroogal.com/product/spendee-app-budget/.

[19] PostgreSQL Global Development Group.
PostgreSQL Documentation, 2024.
URL https://www.postgresql.org/docs/.

[20] Postman, Inc.
Postman, 2025.
URL https://www.postman.com/.

[21] ps.kz.
ps.kz Cloud Hosting, 2024.
URL https://ps.kz.

[22] Redux Team.
Redux Toolkit Documentation, 2024.
URL https://redux-toolkit.js.org/.

[23] SmartBear Software.
Swagger ui, 2025.
URL https://swagger.io/tools/swagger-ui/.

[24] TypeORM Contributors.
TypeORM Documentation, 2024.
URL https://typeorm.io.

[25] YNAB Team.
You need a budget (ynab) features, 2024.
URL https://www.ynab.com/features.